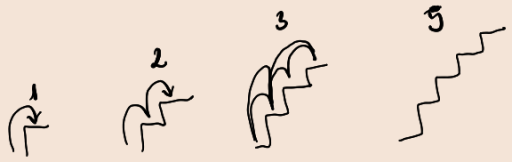
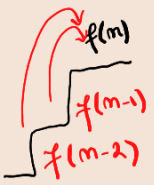
n steps:



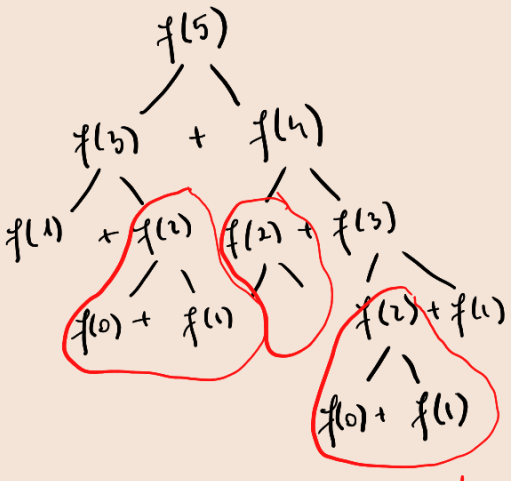


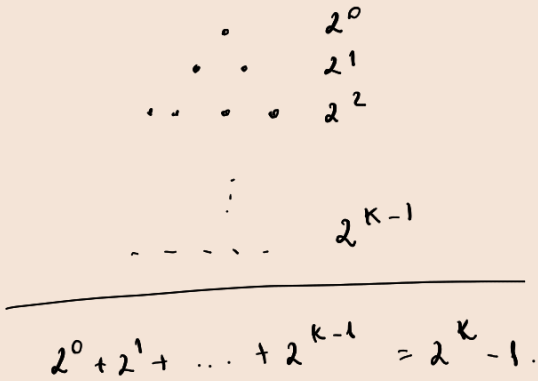
$$\begin{cases} f(0)=0 \\ f(1)=1 \\ f(m)=f(m-1)+f(m-2) \end{cases}$$

$\Leftrightarrow$ m^{th} Fibonacci term.

```
def fibonacci(n):
    if n==0 or n==1
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

$O(2^n)$
upper bound





↳ memoization => logarithmation

```
def fibonacci(n):
    if n==0 or n==1:
        return n

    prev = 0
    .... = 1

    for i in range(n):
        ...
        ...
        ...
    ...
```

→ always does the exact nr. of steps => Θ.

Θ = exactly that nr. of steps

O = could be better

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \times \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \times \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix} \times \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix}$$

$$\times \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 5 \\ 5 & 8 \end{pmatrix}$$

$$A^n \begin{pmatrix} f_{n-2} & f_{n-1} \\ f_{n-1} & f_n \end{pmatrix}$$

$$2^{11} = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2$$

```
def exponentiation (n,p):
    if p==0:
        return 1
    r = exponentiation (p, n//2)
    if ( p %2==0)
        return r*r
    return r*r*n
```

$$T(n) = \begin{cases} 1, & n=0 \\ T\left(\frac{n}{2}\right)+1, & \text{other.} \end{cases}$$

2^k → 2^{k-1}

$$T(2^k) = T(2^{k-1}) + 1$$
$$T(2^{k-1}) = T(2^{k-2}) + 1$$
$$\vdots$$
$$T(2^0) = 1.$$

$\Rightarrow T(2^k) = k$

$\Theta(n)$ logaritmat $\Rightarrow$

$n = 2^k \Rightarrow k = \log_2 n \Rightarrow T(n) = \log_2(n)$

Can we multiply a matrix in the same way? $2^n \rightarrow n$

```
def multiply_matrix (a,b):
    c = [[0,0],[0,0]]
    c[0][0] = ...
    c[0][1] = ...
    c[1][0] = ...
    c[1][1] = ...
```

```
def matrix_exponentiation (m,p):
    if p==n:
        return I2
    r = matrix_exponentiation (m, p//2)
    if p % r == 0
        return multiply_matrix (r,r)
    return multiply_matrix(r, multiply_matrix (m, m))
```

$r * r * m$

```
def fibonacci_2(k)
    if r==0 or r==1
        return n
    m = [[0,1][1,1]]
    r = matrix_exp (m,m)
    return r[1][1].
```

$\rightarrow \Theta(\log_2 n)$

Dynamic programming

Knapsack ~ bara de otel $\Rightarrow$ overlapping

"Introduction to Algorithms."